PROJECT PROPOSAL

Intelligent Noise Reduction & Speech Enhancement System

1. Project Overview

The goal of this project is to design and implement a fully offline speech enhancement system that removes background noise from audio recordings using deep learning. The system will process pre-recorded audio files and output clean, intelligible speech — no internet connection required.

Motivation:

- Background noise degrades the quality of voice recordings and reduces the accuracy of downstream tools such as voice assistants and automated transcription systems.
- Most commercial speech enhancement tools depend on cloud APIs, raising concerns about latency, privacy, and offline usability.
- This project offers a lightweight, beginner-friendly alternative that runs entirely on local hardware.

2. Problem Statement

- Background noise — including street traffic, café ambience, wind, and white noise — significantly reduces speech intelligibility in audio recordings.
- Most speech enhancement solutions require internet connectivity and are not accessible in low-bandwidth or privacy-sensitive environments.
- Transcription and voice assistant accuracy degrades sharply when input audio contains noise.
- A lightweight offline model addresses these issues while keeping the system approachable for students and developers.

3. Proposed Approach

Architecture: Denoising Autoencoder (DAE) with convolutional layers, implemented in TensorFlow/Keras.

Pipeline steps:

- Collect paired (clean, noisy) audio from Mozilla Common Voice and VoiceBank-

DEMAND; synthetically mix noise at varying SNRs.

- Preprocess audio: convert to mono, resample to 16 kHz, normalize amplitude, apply STFT to extract magnitude spectrograms.
- Train DAE: Conv2D encoder (MaxPooling) into Conv2DTranspose decoder (UpSampling); MSE loss; Adam optimizer; batch size 8–16; 20–50 epochs.
- Inference: noisy audio → spectrogram → model prediction → recombine with stored phase → ISTFT → save clean `.wav` file.
- Optional: Tkinter desktop GUI for non-technical users to select files and trigger enhancement.

4. Signal Processing Foundation (Signals & Systems)

In our Signals & Systems course, we studied how the Fourier Transform can break any signal down into its frequency components. This project directly applies that idea to solve the noise problem.

Step 1 — Modelling the Problem

When a noisy recording is made, what we hear is simply the clean speech and the background noise added together:

$$y(t) \;=\; s(t) \;+\; n(t) \tag{1}$$

Here, $y(t)$ is the noisy recording we have, $s(t)$ is the clean speech we want, and $n(t)$ is the unwanted noise. The whole project is essentially trying to recover $s(t)$ from $y(t)$.

Step 2 — Moving to the Frequency Domain

As we learned in class, the Fourier Transform lets us look at a signal in terms of its frequencies instead of time. Because the Fourier Transform is linear, applying it to both sides of the equation above gives:

$$Y(\omega) \;=\; S(\omega) \;+\; N(\omega) \tag{2}$$

This is useful because speech and noise often occupy different frequency regions. For example, human voice is mostly concentrated between 300 Hz and 3400 Hz, while noise like fan hum or white noise is spread across all frequencies. In the frequency domain, separating them becomes a lot more tractable.

Step 3 — Why We Use STFT Instead of a Regular Fourier Transform

A regular Fourier Transform works well for signals that stay the same over time. But speech is not like that — the frequencies change constantly as we move from one word to another. So instead of computing one big Fourier Transform over the whole recording,

we apply it to small overlapping chunks of the audio one at a time. This is called the Short-Time Fourier Transform (STFT):

$$X(\omega,\, \tau) \;=\; \sum_{n} x[n] \cdot w[n-\tau] \cdot e^{-j\omega n} \tag{3}$$

where $w[n]$ is a short window (we slide it along the signal) and τ keeps track of which chunk we are on. The result is a 2D grid called a **spectrogram** — one axis is time, the other is frequency — and it shows us how the frequency content of the speech changes over time.

Step 4 — Magnitude and Phase

Each value in the spectrogram is a complex number. It has two parts:

- **Magnitude** $|X(\omega, \tau)|$ — how strong each frequency is at that moment.
- **Phase** $\angle X(\omega, \tau)$ — the timing offset of that frequency component.

Our model only works on the magnitude (since that is where the noise shows up visually and is easier to learn from). The phase is saved separately from the noisy audio and reused at the end. This is a simplification, but it works reasonably well in practice.

Step 5 — Converting Back to Audio (ISTFT)

Once the model produces a clean magnitude spectrogram, we combine it with the saved phase and apply the Inverse STFT to get the audio back:

$$\hat{s}[n] \;=\; \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{S}(\omega,\, \tau) \cdot e^{j\omega n}\, d\omega \tag{4}$$

This is essentially the inverse of what we did in Step 3 — going from frequency back to time — which is the same idea as the inverse Fourier Transform we covered in class.

How This Connects to Filtering

In class we also studied filters — systems that let some frequencies through and block others. Traditional noise reduction works the same way: figure out which frequencies are noise and reduce them. The classic version of this is the Wiener filter:

$$H(\omega) \;=\; \frac{|S(\omega)|^2}{|S(\omega)|^2 \;+\; |N(\omega)|^2} \tag{5}$$

The problem is that to use this, you need to already know what the noise looks like — which you usually do not. Our deep learning model gets around this by learning from examples instead. It looks at thousands of (noisy, clean) spectrogram pairs during training and figures out on its own how to map one to the other, without needing the noise to be known in advance.

5. Project Timeline

Phase	Weeks	Deliverable / Goal
1 — Setup & Data	1–2	Python environment, dataset download, noise-mixing scripts
2 — Preprocessing	2–3	STFT pipeline, normalised spectrograms (`.npy`)
3 — Model Design	3–4	DAE architecture in Keras (encoder + decoder)
4 — Training	4–5	Model training, hyperparameter tuning, `.h5` export
5 — Inference	5–6	End-to-end pipeline: noisy `.wav` → clean `.wav`
6 — Polish & Demo	6–7	Optional GUI, final report, demonstration

6. Tools & Technologies

Library / Tool	Purpose
Python 3.x	Core programming language
Librosa	Audio loading, STFT, waveform processing
TensorFlow / Keras	DAE model definition, training, inference
NumPy & SciPy	Numerical operations, signal processing
Matplotlib	Waveform and spectrogram visualisation
Tkinter (optional)	Desktop GUI for non-technical users
Mozilla Common Voice	Open-source clean speech dataset
VoiceBank-DEMAND	Paired clean/noisy audio for supervised training

7. Expected Deliverables

- Fully functional offline speech denoising system (end-to-end pipeline).
- Trained model weights saved in `.h5` format.
- Preprocessing and inference scripts (Python).
- Evaluation results: qualitative listening tests and, if feasible, PESQ / STOI metrics.
- Optional: Tkinter desktop GUI.
- Final written report and live demonstration.

8. Anticipated Challenges

- **Audio reconstruction quality** — accurate phase estimation during ISTFT is a known difficulty; will be addressed through iterative testing.
- **Limited local compute** — mitigated by small batch sizes, lightweight model architecture, and incremental training.
- **Dataset preparation** — noise mixing must preserve natural-sounding audio; will use controlled SNR levels and validate outputs perceptually.

9. Future Extensions (Beyond Project Scope)

- Upgrade model architecture to GAN-based or Transformer-based approaches for higher quality output.
- Extend to real-time noise reduction for live microphone input.
- Deploy as a mobile or web application.
- Target edge devices such as Raspberry Pi for embedded audio enhancement.
- Integrate formal evaluation metrics: PESQ (Perceptual Evaluation of Speech Quality) and STOI (Short-Time Objective Intelligibility).